Compilers Experiment Report: Stage 3 - Scope and Block Statements

罗兰 Ruslan | 2021080066

1. Introduction

This report outlines the implementation of block statements in our compiler as part of Stage 3 of the compilers experiment. The primary goal is to enhance the semantic analysis by supporting blocks that define their own scopes.

2. Objectives

- To implement support for block statements in the compiler.
- To manage variable scope effectively, considering shadowing and variable name collisions.
- To update the symbol table construction to accommodate these changes.

3. Background

Block statements allow multiple statements to be grouped together, creating a new scope for variable declarations. This introduces challenges in variable management, particularly:

- **Shadowing:** A variable declared in a block can overshadow a variable with the same name in an outer scope.
- **Non-intersecting scopes:** Variables with the same name can exist in different scopes without conflict.

4. Implementation Details

4.1 Block Statement Structure

Each block statement is treated as a unique scope. The compiler must:

- Recognize the start and end of a block.
- Maintain a stack of symbol tables, where each block's symbol table is pushed onto the stack upon entering the block and popped upon exiting.

4.2 Handling Variable Names

To manage the potential for name collisions, we will:

- Check the symbol table for existing variables in the current scope before adding a new variable.

- Implement a mechanism to allow shadowing where a variable in an inner scope can hide a variable of the same name in an outer scope.

4.3 Symbol Table Updates

The symbol table must be updated to reflect the variable declarations within each block. The process involves:

- Adding variables to the current block's symbol table.
- Implementing checks to ensure that any new declarations do not conflict with existing ones in the same scope.

5. Challenges and Considerations

5.1 Shadowing and Scope Resolution

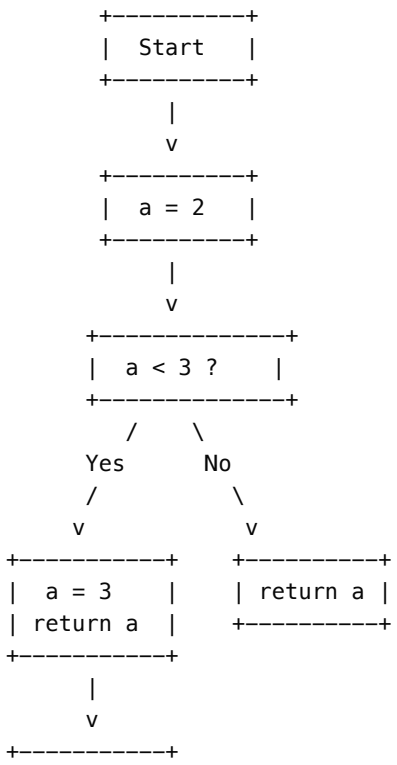
The implementation of shadowing requires careful consideration to ensure that references to variables are resolved correctly based on the scope hierarchy.

5.2 Performance Implications

Maintaining multiple symbol tables and managing scope changes could introduce performance overhead. Optimization strategies may need to be explored.

6. Thinking Questions

Control Flow Graph for the Given MiniDecaf Code



```
| return a  |
+-----------+
```

Explanation

- The program starts by initializing `a` to 2.
- It checks if `a` is less than 3. If true, it enters the inner block, where it initializes a new `a` to 3 and returns it.
- If the condition is false, it returns the original `a`, which is 2.

7. Conclusion

Implementing block statements and managing scope effectively is crucial for the development of a robust compiler. The outlined strategies will aid in addressing the complexities introduced by variable shadowing and scope collisions.